Software Architecture and Design Specification

Event Management System

Version: 1.0

S.No.	SRN	Name
1	PES2UG24CS629	ARYA V D
2	PES2UG24CS902	PRANEETH P SHETTY
3	PES2UG24CS661	SINDHU GV
4	PES2UG24CS900	PREETHI T

Project: A web-based system to manage and organize events with participant registrations.

07 October 2026

Revision History

Version	Date	Author	Change Summary
1.0	07-10-2026	Project Team	Initial Software Architecture and Design Specification for the Event Management System.

Approvals

Role	Name	Signature/Date
Course Coordinator	Dr.Pradeep Kumar	
Project Team	ARYA V D; PRANEETH P SHETTY; SINDHU GV; PREETHI T	

1. Introduction

1.1 Purpose: This document specifies the software architecture and detailed design of the Event Management System. It translates the requirements in the SRS into a logical architecture, components, interfaces, data responsibilities, security controls and key interaction flows.

1.2 Scope: The design covers user registration and authentication, role-based authorization, event creation/editing/publishing/cancellation, event discovery/search/filtering, participant registration/cancellation/history, organizer registration management, administrator management, notifications and audit records. Version 1.0 excludes payment processing, specialized QR/barcode hardware, external calendar synchronization and production email/SMS infrastructure.

1.3 Audience: Developers, testers, project members, course instructors/evaluators and future maintainers.

1.4 Definitions: EMS - Event Management System; RBAC – Role-Based Access Control; REST API – HTTP-based interface between frontend and backend; Registration – participant's association with an event after successful registration; Audit Record – timestamped record of an important administrative or security-sensitive action.

2. Document Overview

2.1 How to use this document: Sections 3 and 4 describe the architecture and design of the system. The component diagram shows the main logical modules and their interactions. The sequence diagrams show representative authentication and event-registration flows. API, security, error-handling and traceability sections provide implementation guidance.

2.2 Related Documents: Event Management System SRS Version 1.0 dated 12-09-2026; Software Test Plan Version 1.0 dated 16-09-2026; Requirements Traceability Matrix in the SRS; supplied SAD template.

3. Architecture

3.1 Goals & Constraints

- Goals: modularity, secure role-based access, consistent registration data, usable participant/organizer workflows, maintainability and the ability to support additional users/events later.
- The architecture shall preserve clear separation between UI, REST API handling, business rules/authorization and persistent data.
- Constraints: React.js shall be used for the frontend; Node.js/Express.js for the backend; MongoDB for persistent application data; server-side validation and authorization are required; a shared version-control repository is used.
- Version 1.0 is a mini-project web application demonstrated locally on team computers using a modern browser such as Chrome; no specialized hardware is required.

3.2 Stakeholders & Concerns

Stakeholder	Primary concern
Participant	Simple event discovery, clear details, short registration flow and protection of registration information.
Organizer	Accurate event lifecycle, capacity/deadline handling and correct participant information for owned events.
Administrator	Controlled system-level user/event management and audit information.
Developers / Project Team	Modularity, shared repository, clear interfaces, maintainability and manageable mini-project scope.
Testers / Evaluators	Requirement traceability, verifiable behavior, security, performance and acceptance-test evidence.

3.3 UML Class Diagram

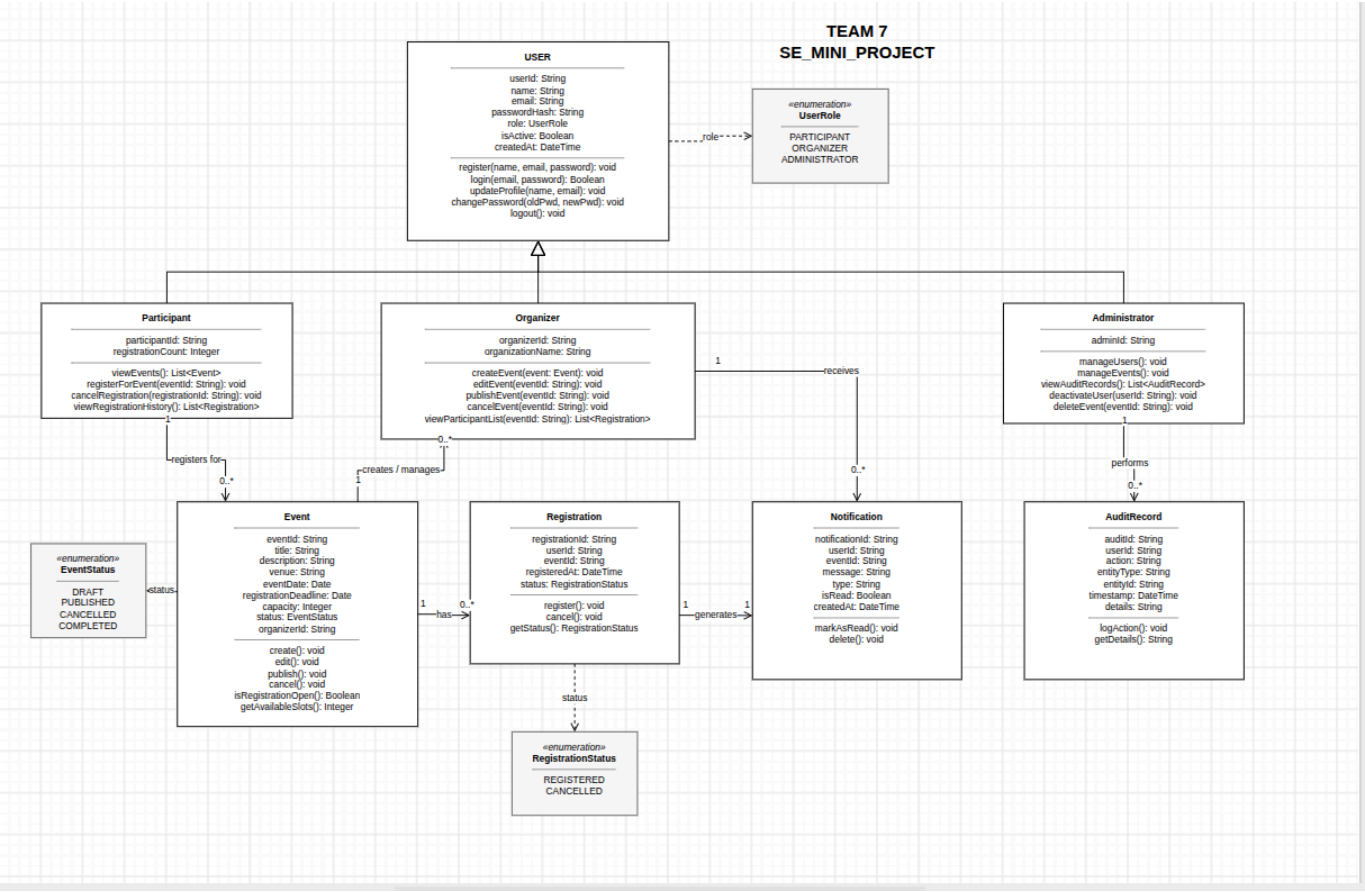


Figure 1: UML Class Diagram of the Event Management System.

3.4 Component Descriptions

Component	Responsibility
React.js Frontend	Provides registration/login, event listing, search/filter, event details, organizer event-management screens, participant registration/history and administrator screens. Performs client-side interaction and displays validation/error feedback.
REST API Layer (Node.js/Express.js)	Receives HTTP requests, routes them to application operations, returns JSON responses and provides the boundary between frontend and backend.
Authentication & Authorization	Authenticates registered users and enforces protected access according to Participant, Organizer and Administrator roles. Authorization checks are server-side.
Event Management	Creates, edits, publishes and cancels events; validates event lifecycle rules and exposes event discovery/details operations.
Registration Management	Checks authentication, deadline, capacity and duplicate-registration rules; creates/cancels registrations and updates registration status/capacity.
Administration & Audit	Handles permitted user/event management and records important administrative/security-sensitive actions with timestamps.

MongoDB Data Store	Persists users, events, registrations and audit records as required by the SRS.

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: Layered architecture with a web client, REST API/application layer, business-rule modules and a persistence layer. This follows the SRS product perspective: React.js communicates with Node.js/Express.js REST APIs, while the backend applies business rules and accesses MongoDB.

Rationale: The layered structure separates presentation, transport, business rules/authorization and persistence. It supports maintainability and traceability without introducing the operational complexity of a distributed microservices architecture, which is unnecessary for the stated mini-project scope. This is a design decision for Version 1.0, not a requirement that prevents future decomposition.

3.6 Technology Stack & Data Stores

Layer	Technology / design
Frontend	React.js
Backend	Node.js with Express.js
Communication	HTTP-based REST APIs; JSON request/response data
Persistence	MongoDB
Browser	Modern supported browser such as Chrome
Deployed security	HTTPS for deployed environments handling real credentials
Version control	Shared repository as required by the SRS

3.7 Risks & Mitigations

Risk	Mitigation
Frontend/backend contract mismatch	Keep REST interfaces explicit; validate request/response structures through integration/API testing.
Unauthorized role access	Perform server-side RBAC checks for every protected Organizer/Administrator operation.
Duplicate or over-capacity registration	Validate deadline, capacity and duplicate active registration before persistence; verify data integrity with edge/concurrency tests.
Backend/database unavailability	Use a reproducible local setup and verify frontend/backend/database startup before testing.
Cold-start delay in hosted environments	Treat the documented first-request delay as a known limitation and exclude it from the warm-response performance targets, as specified in the SRS.

3.8 Traceability to Requirements

Requirement	Design element
EMS-F-001–004	Authentication component + React registration/login UI
EMS-F-005–008	Event Management component
EMS-F-009–012	Event Discovery/Search/Filter component
EMS-F-013–017	Registration Management component + MongoDB registration data
EMS-F-018–020	Organizer Registration Management
EMS-F-021–023	Administration component + RBAC

EMS-F-024–027	Notification/confirmation and Audit components
EMS-NF-001–003	REST API and modular backend operations; measured through performance tests
EMS-NF-004	Registration business rules + MongoDB persistence
EMS-NF-005	React.js responsive workflow/UI design
EMS-NF-006	Application/security event logging and audit records
EMS-SR-001	Authentication data model and salted one-way password hashing
EMS-SR-002 / EMS-SR-006	Server-side authorization / RBAC middleware
EMS-SR-003	Server-side input validation
EMS-SR-004	Safe API responses and logs
EMS-SR-005	Timestamped security audit records

3.9 Security Architecture

Security objectives: The architecture addresses confidentiality, integrity and controlled access as stated in the SRS.

Threat / concern	Architectural control
Spoofing / unauthorized access	Authentication before protected functions; role-based authorization.
Tampering / invalid data	Server-side input validation; business-rule validation before persistence.
Information disclosure	Do not expose passwords, authentication tokens, secrets or internal stack traces in normal responses/logs; use HTTPS in deployed environments handling real credentials.
Privilege escalation	Server-side RBAC checks; Administrator-only operations are inaccessible to Participant and Organizer roles.
Registration integrity	Check deadline, capacity and duplicate active registration before creating a registration; verify persistent state.
Auditability	Timestamp important administrative/security-sensitive actions; audit records must not contain passwords or authentication secrets.

4. Design

4.1 Design Overview

Design overview: The design separates the system into presentation, REST API, application/business-rule and persistence responsibilities. Core domain modules are Authentication, Event Management, Registration Management, Administration and Audit/Notification handling. This structure supports the SRS quality attributes of usability, reliability, maintainability, scalability and security.

4.2 UML Sequence Diagrams

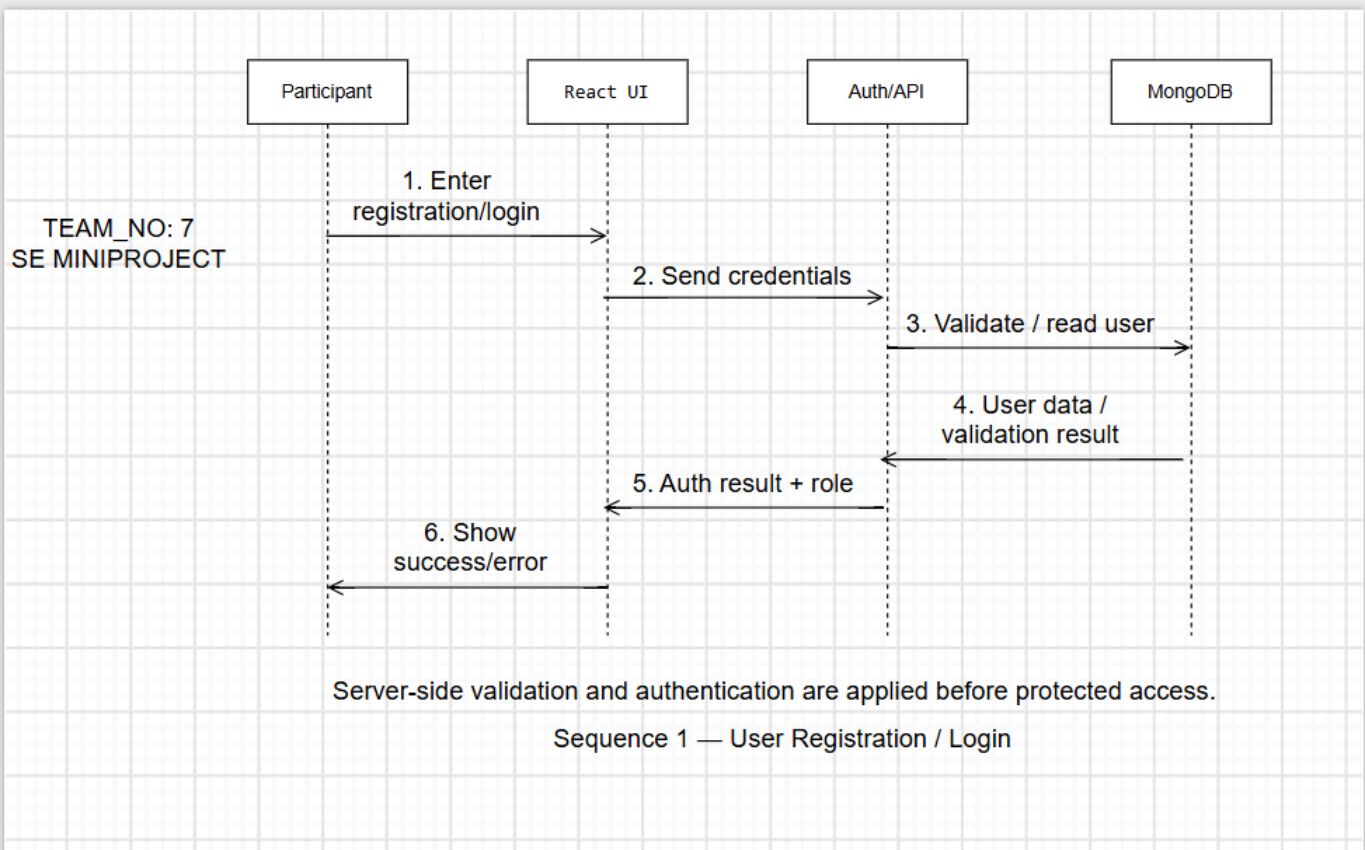


Figure 2: User registration/login sequence.

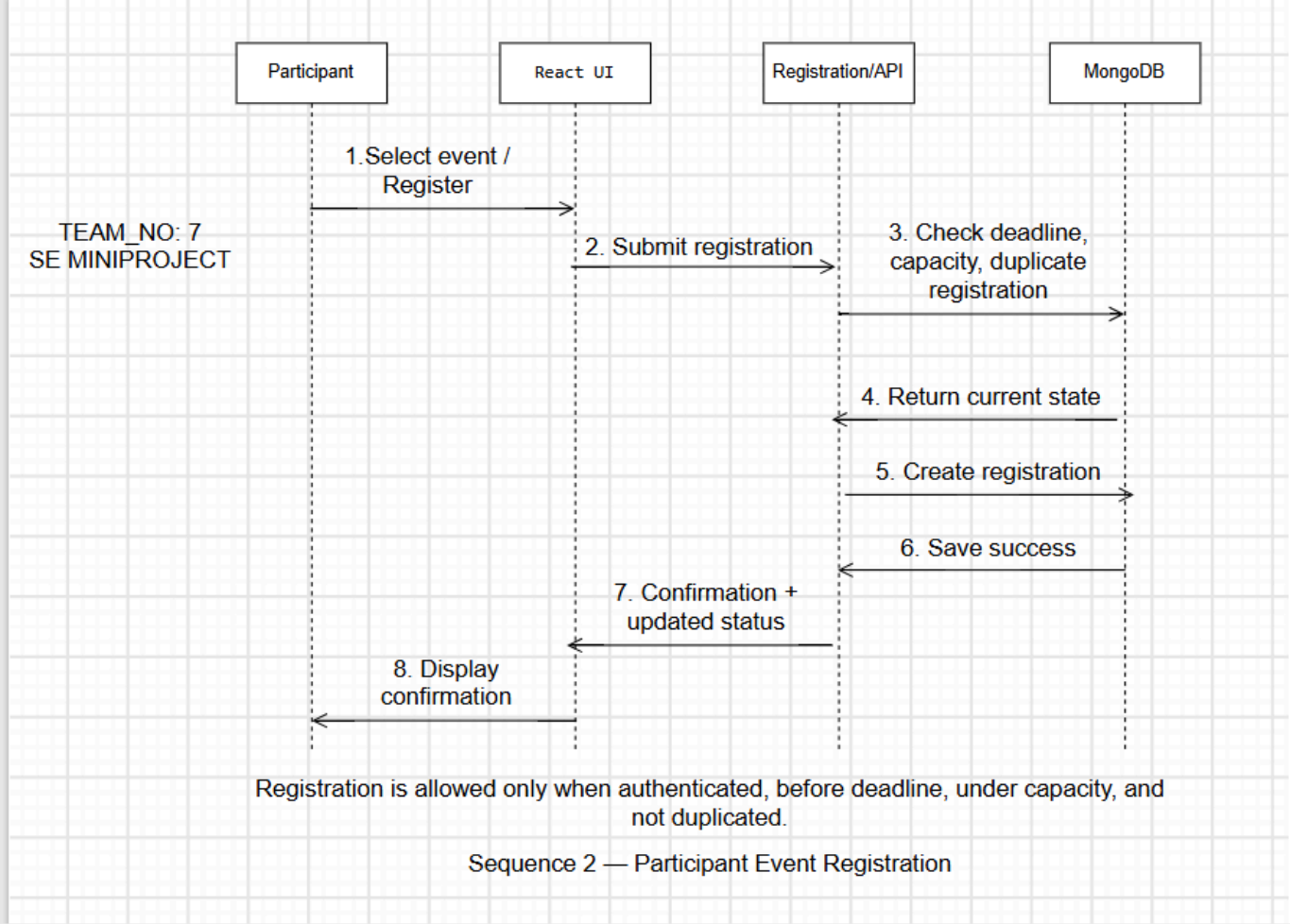


Figure 3: Participant event-registration sequence.

4.3 API Design

Design note: The SRS specifies REST APIs using JSON but does not prescribe exact endpoint names. The following endpoint names are therefore proposed interface names for implementation and should be kept consistent across frontend, backend and testing artifacts.

Component / Endpoint	Method	Request	Response / Errors
Authentication - /api/auth/register	POST	{ name, email, password, role }	Success: created-user/status; Errors: validation failure, duplicate account identifier.
Authentication - /api/auth/login	POST	{ email, password }	Success: authentication status + role/session information; Error: invalid credentials.
Events - /api/events	POST	Event details such as title, description, schedule, venue/online info, capacity, deadline	Success: created event; Errors: validation failure, unauthorized organizer.
Events - /api/events/search	GET	Query/filter parameters such as title, category, date	Success: matching published events; Error: invalid query parameters.

Registration - /api/events/{eventId}/register	POST	Authenticated user context / registration request	Success: registration confirmation + updated status; Errors: unauthorized, deadline closed, capacity reached, duplicate registration.
Registration - /api/events/{eventId}/cancel	POST	Authenticated participant context	Success: cancellation confirmation + updated capacity/status; Errors: unauthorized or cancellation not permitted.
Organizer - /api/events/{eventId}/registrations	GET	Authenticated organizer context	Success: count + authorized participant list; Error: event not owned / unauthorized.
Admin - /api/admin/users or /api/admin/events	GET/POST/PATCH	Administrator-authorized management request	Success: management result; Error: forbidden for non-Administrator roles.

4.4 Error Handling, Logging & Monitoring

- Return clear validation, authentication, authorization, conflict and server-error responses without exposing internal implementation details.
- Use consistent JSON response structures across REST endpoints.
- Do not place passwords, authentication secrets, tokens or internal stack traces in normal logs or responses.
- Record important administrative and security-sensitive actions with timestamps.
- Security-relevant events include repeated authentication failures and privilege changes.
- Operational evidence shall support troubleshooting and audit review, consistent with EMS-NF-006 and TC-OPS-01.

4.5 UX Design

- Participant flow: register/login → browse/search/filter → view event details → register → receive confirmation → view/cancel registration where permitted.
- Organizer flow: login → create/edit event → publish/cancel → view registration count and authorized participant list.
- Administrator flow: login → access permitted user/event management → review relevant audit information.
- Forms shall provide clear validation and error feedback.
- Core participant and organizer workflows shall remain usable on representative desktop and mobile browser viewports.
- Version 1.0 does not include internationalization.

4.6 Open Issues & Next Steps

- Finalize exact REST endpoint names and JSON schemas during implementation while maintaining SRS behavior and traceability.
- Define the concrete MongoDB document/collection schema and indexes required for users, events, registrations and audit records.
- Define the exact authentication/session mechanism used by the implementation; it must satisfy the SRS security requirements.
- Complete component and sequence diagrams alongside implementation changes so the SAD remains synchronized with the code.

- Future enhancements may include features outside Version 1.0 only through a documented requirements change; payment processing, external calendar synchronization and production email/SMS are currently out of scope.

5. Appendices

5.1 Glossary

Term	Definition
React.js	Frontend JavaScript library used for the user interface.
Node.js	JavaScript runtime used for backend application services.
Express.js	Backend web framework used to implement REST APIs.
MongoDB	Database used for persistent application data in Version 1.0.
REST API	HTTP-based interface through which frontend and backend exchange application data.
RBAC	Role-Based Access Control; permissions are assigned according to user role.
Audit Record	Timestamped record of an important administrative or security-sensitive action.
SRS	Software Requirements Specification.
STP	Software Test Plan.
RTM	Requirements Traceability Matrix.

5.2 References

- Event Management System - Software Requirements Specification, Version 1.0, 12-09-2026.
- Event Management System - Software Test Plan, Version 1.0, 16-09-2026.
- SAD Template - Software Architecture and Design Specification (supplied template).

5.3 Tools

- React.js for frontend implementation.
- Node.js / Express.js for backend REST APIs.
- MongoDB for persistent application data.
- A modern web browser such as Chrome for UI execution and testing.
- REST API client such as Postman or equivalent for API verification, as planned in the STP.
- UML diagramming/design tools such as draw.io.